Does the optimizer do what it says? A black-box audit of evaluation integrity in 21 Python optimization libraries

DANIJEL JOVANOVIĆ, University of Novi Sad, Faculty of Technical Sciences, Serbia

Comparisons of black-box optimizers assume that each optimizer evaluates the cost function as its documentation says and reports what it actually found. In engineering applications such as power-flow-based dispatch, every hidden, repeated or out-of-budget evaluation costs real compute, and every inconsistent report can change a comparison. We present a library-independent black-box audit that records every call to the cost function and checks call-level integrity (out-of-box, over-budget and post-return evaluations, repeated points), the consistency of the reported result with the recorded calls, and reproducibility under a fixed seed within one process and between fresh processes. The audit is validated on planted faults. Across 21 Python libraries in two preregistered studies (nine popular libraries chosen by us and twelve selected by a deterministic external rule), with one prespecified configuration per library, integrity findings other than corner evaluations occurred in 8 of 21 tested configurations and behaviour contradicting the library's own documentation in 4 of 21. On a real AC economic-dispatch cost function (pypower, MATPOWER cases), the library-level detection status agreed with the test-function studies for 19 of 21 libraries, and 7 of the 8 libraries with a finding showed one again. The real cost function also exposed behaviour the test functions missed: SciPy `differential_evolution` discards an improving polished point when L-BFGS-B reports `success=False`. We diagnosed and disclosed five issues with minimal reproducers, three of which we did not find reported in a documented search: seed dependence of SMAC 2.4.1 on `PYTHONHASHSEED`, a sign flip in the reported cost of Gradient-Free-Optimizers 1.13.0, and the SciPy polishing behaviour. A corrected, preregistered rerun with recorded hash seeds and full call traces confirmed both cross-process reproducibility findings and reproduced 318 of 358 archived call sequences bit for bit; the remaining differences are explained by hash seeds and by the number of BLAS threads. A preregistered "corner evaluation" measure proved non-specific on test functions; we report this negative result. We recommend running such an audit, including a cross-process reproducibility check, before comparing optimizers on an expensive cost function.

ACM Reference Format:
Danijel Jovanović. 2026. Does the optimizer do what it says? A black-box audit of evaluation integrity in 21 Python optimization libraries. *ACM Trans. Evol. Learn. Optim.* 1, 1 (October 2026), 14 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Benchmarking practice in evolutionary computation counts function evaluations carefully: COCO [Hansen et al. 2021] and IOHexperimenter [de Nobel et al. 2024] wrap the objective and record every call. Practitioners who compare optimizers on their own expensive models rarely use such platforms. They call libraries directly through the library's own budget, seed and result interfaces, and trust that:

1. the budget parameter bounds the number of evaluations;
2. every evaluation happens inside the declared box and during the search;

Author's Contact Information: Danijel Jovanović, University of Novi Sad, Faculty of Technical Sciences, Novi Sad, Serbia, danijel.jovanovic@uns.ac.rs.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 2688-3007/2026/10-ART
https://doi.org/10.1145/nnnnnnn.nnnnnnn

3. the returned solution and cost are the best evaluated pair;
4. a fixed seed reproduces the run.

When any of these fails silently, the comparison is wrong in a way that no statistical test reveals.

The work started from a concrete case. In our own library, GAdapt, input validation evaluated the cost function at the all-lower and all-upper corners of the box. On a binary route-choice problem, the all-lower corner is the trivial first-path solution. Most likely, these two hidden evaluations made a validation probe look like a search result in an earlier experiment (Section 7.1). The original raw data of that experiment are no longer available, so this remains the most plausible reading, not a verified fact. This raised a general question: how often do optimization libraries evaluate, count, reproduce or report differently from what their users assume?

Contributions:

1. A black-box, library-independent audit with explicit call classes and result-consistency checks. Its cross-process reproducibility check (D3x) adapts known hash-order randomization to the seed contract of optimization libraries and exposes seed failures that within-process checks miss.
2. Validation of the audit on planted faults, including faults that are only visible across processes.
3. Two preregistered prevalence studies on 21 Python libraries, one of them drawn by an external sampling rule, and a third study that adds cross-process reproducibility to the first sample.
4. A preregistered transfer study on a real infrastructure cost function (AC economic dispatch) for all 21 libraries.
5. Five diagnosed issues with minimal reproducers, causal patch tests where applicable, and disclosure to the maintainers.
6. A negative methodological result: a natural "probe" measure is not specific on test functions, which changes the prevalence estimate.
7. A practitioner command-line tool with a six-step checklist.

2 Related work

- **Evaluation accounting and benchmarking platforms.** COCO [Hansen et al. 2021] and IOHexperimenter [de Nobel et al. 2024] count evaluations and log performance. IOHexperimenter can also wrap an arbitrary user callable (`ioh.wrap_problem`), so wrapping and counting a user objective is established. These platforms log what the optimizer evaluates; they do not check the reported result against the recorded calls, compare behaviour with the library's documented contract, or test reproducibility across processes.
- **Invariance and structural bias.** Kudela [2022] showed that many metaheuristics exploit the centre of the search space. BIAS [Vermetten et al. 2022] detects structural bias from final positions. Invariance testing is an established principle [Hansen et al. 2011]. We use invariance only as a secondary property and claim no novelty for it.
- **Comparing frameworks.** Ravber et al. [2025] compared six algorithms across 13 frameworks, found large performance differences between implementations of the "same" algorithm, and analysed source code, including redundant fitness evaluations, missing evaluation-based stopping and implementation defects. Our audit is complementary: it is black-box, classifies every call, and checks reported results and the documented seed contract.
- **Metamorphic testing** has been applied to genetic algorithms [Rounds and Kanewala 2018] and to industrial optimisation software [Duque-Torres et al. 2025a,b]. It checks relations between runs and is closest in spirit to our seed and documentation checks. Its targets are

algorithm or application properties, not the evaluation contract between a library and its user.

- **Nondeterminism from iteration order.** Hash-based iteration order is a known source of flaky tests. NonDex [Gyori et al. 2016] randomizes the iteration order of Java collections to expose code that relies on an unspecified order, and Gruber et al. [2021] studied flaky tests in Python. Our D3x applies the same idea, varying `PYTHONHASHSEED` between two fresh processes, to the documented seed contract of optimization libraries. We claim no novelty for the technique.

Our claim is narrow. No component (wrapping and counting, invariance, seeding, hash-order randomization) is new in isolation. The contribution is the integration into one validated, library-independent black-box audit that classifies every call, checks reported results against recorded calls and documented relations, including cross-process reproducibility, together with the empirical diagnoses it produced. "Previously unreported" in this paper means not found in our documented search of each project's issue tracker and changelog (23 October 2026).

3 The audit

A recorder wraps the user's cost function and stores, for every call, the point x, the value f and a phase: search, or post, meaning after the optimizer's main call returned. Each library is run through a small adapter that uses only its documented interfaces for budget, seed, bounds and result.

Class	Definition	Integrity?
C1	x equals the all-lower or all-upper corner of the box	preregistered as integrity; see Section 6.3
C2	x bit-identical to an earlier evaluated point	waste (reported as a share)
C3	some coordinate of x outside [lower, upper]	yes
C4	more calls than a documented maximum (only where one is documented)	yes
C5	calls after the optimizer's main call returned	yes
R1	reported x^* was never evaluated (every coordinate within 1e-12)	yes
R2	reported f^* differs from the recorded $f(x^*)$: $\lvert f^* - f(x^*)\rvert >$ 1e-9 $\cdot \max(1, \lvert f(x^*)\rvert)$	yes
R3	reported f^* worse than the best search-phase value b: $f^* - b >$ 1e-9 $\cdot \max(1, \lvert b\rvert)$	yes
D2	reported evaluation count differs from the recorded count	yes
D3	same seed, different call sequence on a rerun in the same process	yes

Class	Definition	Integrity?
D3x	same seed, different call sequence in two fresh processes (see the note on hash seeds below)	yes

The call sequence is compared through a SHA-256 hash over (x, f, phase) for all calls. Exact predicates, phase boundaries and applicability rules are in Appendix A.

Note on hash seeds (correction). The protocols intended D3x to compare two processes with `PYTHONHASHSEED` 1 and 2. The Study 2–4 launchers started the library interpreter with `python -I`, which implies `-E` and ignores `PYTHONHASHSEED`. Each process therefore had Python's default, independent per-process hash randomization. D3x as run compares two fresh processes with independent random string hashing; it detects the same class of nondeterminism, but the controlled same-seed/different-seed comparison was not performed by the audit. The SMAC attribution to `PYTHONHASHSEED` first rested on a separate reproducer run without `-I` (Section 6.2), which we re-ran: `PYTHONHASHSEED=1` twice gives the identical incumbent, 2 and 3 give different ones. Study 5 (Section 6.5) repeated D3x with effective, recorded hash seeds and confirmed both findings within the audit. R1–R3 are not applicable when a library returns no result, as in ask-and-tell interfaces. For libraries that work in float32, the recorder stores the value the library actually receives, so that rounding is not counted as R2.

Test problems:

- shifted sphere, with the optimum at 0.37 of the range;
- Rastrigin;
- the floor plateau $\Sigma\lfloor x_i \rfloor$;

in $N \in \{2, 10\}$, on the box $[-5, 5]^N$, with seeds 6000–6004. This gives 30 cells per library.

4 Validation of the audit

Before each study, a reference random search was run with exactly one planted fault per mutant. Faults covered:

- a corner probe;
- a re-evaluated duplicate;
- one call over the budget;
- reporting an unevaluated point;
- reporting a wrong value;
- seeding from `hash()`, visible only across processes;
- seeding from the clock.

The audit had to flag exactly the planted class and nothing on the clean reference. GAdapt with its default input validation had to produce C1, and without it had to produce none. All checks passed: 8/8 in Study 1 and 11/11 in Study 2. The practitioner tool repeats these checks in its test suite.

5 Studies

Both protocols, including the decision thresholds and the documentation-classification rule of Study 2, were committed before any code was written.

Study 1 (I49). Nine popular libraries:

- pymoo GA;

- DEAP `eaSimple`;
- PyGAD;
- geneticalgorithm;
- Nevergrad `NGOpt`;
- Optuna TPE;
- PySwarms;
- cma `fmin2`;
- SciPy `differential_evolution`.

Settings: one prespecified configuration per library (mostly defaults; operator or population choices where the API requires them, frozen in the protocol), about 1000 evaluations, set through the documented parameter. GAdapt served as a positive control.

Study 2 (I50). A new sample drawn by an external rule from a frozen snapshot:

- **Source A:** GitHub repositories in Python with at least 300 stars and one of 12 optimization topics.
- **Source B:** the Python frameworks in Ravber et al. [2025].
- **Eligibility:** general-purpose library, not archived, installable on Python 3.14 with NumPy 2.x, documented box-constrained single-objective API, and a smoke test with a constant cost function.

Of 62 candidates, 12 were eligible: baybe, bayes_opt, bofire, cmaes, emukit, EvoloPy, evotorch, evox, Gradient-Free-Optimizers (GFO), NiaPy, scikit-opt and SMAC. The budget was 200 evaluations. Before the run, we froze the configurations and the documentation quotes for budget, seed, result and bounds.

Study 3 (I51). D3x on the Study 1 sample, with the unchanged Study 1 adapters.

Study 4 (I52). All 21 libraries, with unchanged adapters and budgets, on a real infrastructure cost function: AC economic dispatch with fixed voltages.

- **Cost function:** pypower `runpf` on MATPOWER case14, case39 and case118 (4, 9 and 53 variables). The cost is generation cost plus penalties for voltage, slack, reactive and branch-loading violations; 2–16 ms per evaluation.
- **Seeds:** 5 per case.
- **Preregistered hypothesis:** the library-level status ($\geq$ 1 integrity finding, C1 excluded) agrees with the synthetic studies for $\geq$ 17/21 libraries.
- **Deviation:** the first execution failed at import in every cell, before any evaluation (a missing dependency and a pypower version incompatible with NumPy 2.5). It was archived, the environment was fixed, and the study was run once.

Study 5 (I55, correction). A preregistered rerun of all test-function cells of Studies 1–2 (21 libraries $\times$ 30 cells), with the archived package versions, that fixes the hash-seed error (Section 3) and archives full call traces.

- **Processes per cell:** A (`PYTHONHASHSEED=1`, plus a rerun in the same process), A′ (`PYTHONHASHSEED=1`) and B (`PYTHONHASHSEED=2`), launched with `-s` and `PYTHONSAFEPATH=1`. Each process reports a sentinel string hash; a cell counts only if A and A′ agree and A and B differ (manipulation check).
- **Measures:** D3 (A vs its rerun), D3x-same (A vs A′) and D3x-diff (A vs B). All classes are recomputed from the archived trace of A and compared with the archived Study 1–2 flags.
- **Preregistered hypotheses:**
  - H55a: D3x-diff occurs exactly for {Nevergrad, SMAC};
  - H55b: SMAC shows no D3x-same and D3x-diff in $\geq$ 25/30 cells;

  - H55c: Nevergrad shows D3x-same, which would mean its cause is not the hash seed;
  - H55d: libraries without D3/D3x reproduce their archived flags.
- **Deviations (recorded during the run):**
  - O1: the first 10 libraries ran an unneeded extra rerun in A′ and B; this did not change the measured quantities.
  - O3: the driver omitted `VECLIB_MAXIMUM_THREADS=1` (Apple Accelerate BLAS), which Studies 1–2 had set.

6 Results

6.1 Prevalence

Outcome	Study 1	Study 2	Pooled
≥ 1 finding, all classes (preregistered)	4/9 (14–79%)	11/12 (62–100%)	15/21 (48–89%)
≥ 1 finding excluding C1 (post hoc, Section 6.3)	4/9	4/12 (10–65%)	8/21 (18–62%)
≥ 1 finding excluding C1 and design-caused C3 (post hoc)	3/9	3/12	6/21 (11–52%)
Behaviour contradicting the library's documentation	2/9	2/12 (2–48%)	4/21 (5–42%)

Clopper–Pearson 95% intervals are in parentheses. They describe binomial uncertainty conditional on treating the tested configurations as exchangeable draws; Study 1 is a purposive sample and Study 2 a deterministic rule, so the intervals are model-dependent summaries, not estimates for a defined population of libraries. Each library was tested in one configuration, which does not characterize every algorithm it contains.

Cross-process reproducibility on all 21 libraries. Study 3 found D3x in 1 of the 9 Study 1 libraries: Nevergrad, in the same cells as D3. Over both samples, a fixed seed fails to reproduce the run for 2 of 21 libraries, Nevergrad and SMAC. The SMAC case is visible only across processes.

Waste. The median share of repeated evaluations was 47% for scikit-opt DE, 41.5% for geneticalgorithm and 11.5% for DEAP. With an expensive cost function, that is close to half of the budget.

6.2 Findings and diagnoses

Library	Finding	Cause	Status
DEAP	evaluations outside the box (30/30 runs)	bounds are the user's responsibility	design; previously noted [Ravber et al. 2025]
PyGAD	21 evaluations after return	`best_solution()` without `pop_fitness` re-evaluates the population	documented

Library	Finding	Cause	Status
cma	up to 1011 evaluations for `maxfevals` = 1000	the maximum is checked per iteration	known (pycma #195)
Nevergrad	`NGOpt` not reproducible under a fixed seed (D3, and D3x from the first evaluation)	`cma.fmin` called without a seed (pycma default `seed='time'`); the metamodel's inner optimizer is created without a seeded parametrization	known issue (#1572, open since 2023); **root cause and a 12-line patch are ours**
SMAC 2.4.1	D3x in 30/30 runs, no D3	`LocalSearch._get_initial_points` returns `list(set(...))` of `Configuration`, whose hash is a string hash	not found in documented search; reported (#1380)
GFO 1.13.0	R2 in 25/30 runs, R3 in 10/30	with `optimum="minimum"`, the internal negation is not undone in `best_score` or `search_data`	not found in documented search; reported (#106)
scikit-opt DE	R3 in 1/30 synthetic and 2/15 dispatch runs; 10/200 seeds on a 10-D sphere; 301 calls for a nominal 200	after selection, `self.Y` holds only trial values, so a retained parent better than all trials is never seen as best; parents are re-evaluated every generation	a title-only report existed (#187); **cause diagnosed, fix tested (10/200 → 0/200)**, reported (#248)
SciPy DE 1.18.1	R3 in 5/15 dispatch runs, none on test functions; in the disclosure reproducer (case118) the returned value was 89% worse than an evaluated point	an improving polish is accepted only if L-BFGS-B reports `success=True`	behaviour not described in the documentation (a documentation gap, not a contract violation); not found in documented search; reported (#26479); reproduced on the nightly build; a SciPy maintainer confirmed the behaviour is intended and proposed a docstring change (2026-10-23)

Library	Finding	Cause	Status
evotorch SNES	evaluations outside the box	SNES does not support hard bounds; the README uses `initial_bounds`	design
bayes_opt	on the plateau, 7 of 200 requested evaluations	a repeated suggestion returns a cached value but consumes an iteration	documented in `TargetSpace.probe`

Causal checks:

- **Nevergrad:** with our patch, the full call sequence was identical within and across processes in 16/16 cells, against 0/6 unpatched. The issue's original reproducer became deterministic.
- **SMAC:** order-preserving de-duplication made the sequence independent of `PYTHONHASHSEED`. With default hash randomization, three runs with `seed=0` gave three different incumbents.
- **GFO:** reproduced on the development branch; the proposed fix was not tested.

6.3 A negative result: corner evaluations are not specific

C1 was designed to catch input-validation probes such as GAdapt's. In Study 2 it fired for nine libraries, almost always for legitimate reasons:

- clipping to the bounds in particle swarms;
- acquisition optima at the boundary in Bayesian optimization;
- deliberate vertex initialization (GFO `initialize={"vertices": 4}`);
- on the plateau, the lower corner is the region of the global optimum.

No Study 1 library had a C1-only finding, so the Study 1 estimate is unaffected.

The preregistered Study 2 verdict ("common", 11/12) is therefore driven by a non-specific class. Excluding C1, which is a post hoc analysis, gives 4/12. Under the same thresholds this would be inconclusive: the lower bound of 9.9% is just below the 10% threshold.

We report both and treat the C1-free estimate as the defensible one. The practitioner tool reports corner evaluations as information, not as a problem.

6.4 Transfer to a real infrastructure cost function

Hypothesis supported, in the preregistered sense. The library-level detection status (≥ 1 finding, C1 excluded) agreed for 19 of 21 libraries (95% CI 70–99%). Agreement includes absence: the confusion matrix (test functions $\times$ dispatch) is 7 positive/positive, 12 negative/negative, 1 positive/negative (Nevergrad) and 1 negative/positive (SciPy DE). Of the 8 libraries with a finding on test functions, 7 showed one again on dispatch. Completion was 295 of 315 dispatch cells (Nevergrad 10, bayes_opt 5 and bofire 5 cells failed), which limits the evidence for negative status of those libraries; dispatch also offered 15 cells per library against 30 synthetic.

- **Findings that transferred:** 7 of the 10 (library, class) findings from the test functions reappeared: DEAP C3, PyGAD C5, cma C4, evotorch C3, GFO R2, scikit-opt R3 and SMAC D3x.
- **Findings that did not transfer:**
  - Nevergrad's D3 had appeared only on the 10-D plateau, and on dispatch 10 of its 15 cells crashed with a NumPy 2.5 error.

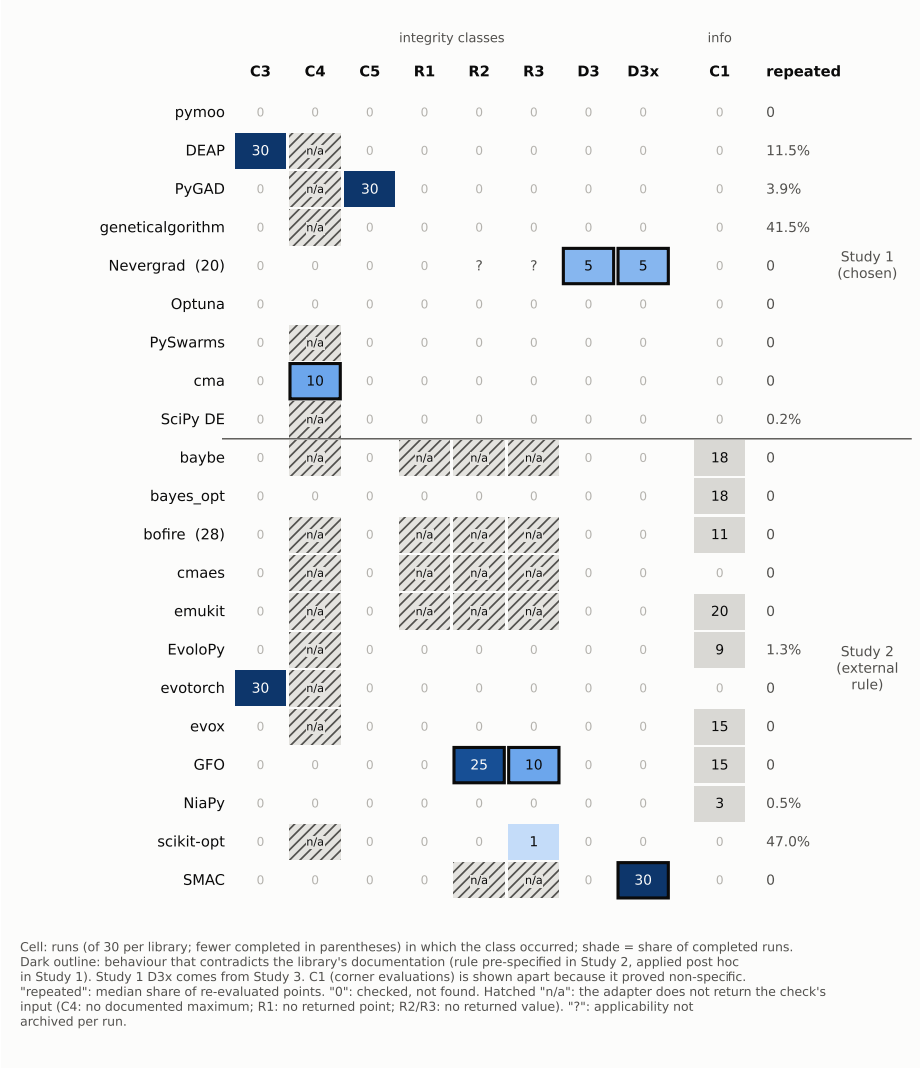


Fig. 1. Runs with each finding class, per library, in the test-function studies (1–3). Dark outlines mark behaviour that contradicts the library's documentation (a curated input of `make_figure.py`, frozen with the Study 2 quotes). C1 is shown apart (Section 6.3). D2 is not shown: it was never flagged. A grey "0" is a check that was made and found nothing; hatched "n/a" cells are checks whose input the library's adapter does not return (read from the adapter code and the archived `r_applicable`), and "?" marks Nevergrad's R2/R3, which apply only in runs whose recommendation carries a loss (not archived per run). The right column gives the median share of repeated evaluations. Generated by `make_figure.py` from the raw outputs.

  - GFO's R3 on the test functions is plausibly a side effect of the sign flip on negative plateau values.
- **One new finding, only on the real cost function:** SciPy DE, in 5 of 15 runs. On case118, L-BFGS-B polishing used 15,444 evaluations, stopped at its `maxfun` limit and reached 143,102; the returned value was 271,061. The same behaviour appears on a 10-D quadratic with a

penalty wall, where the returned value was 22.97 against an evaluated 1.10, and on the SciPy nightly build.

- **Total evaluations for SciPy DE:** 1,270–16,239 for a nominal DE budget of 795–960. The documented formula excludes polishing.
- **C1 on dispatch:** it fired for the GAdapt control in 15/15 runs and less often otherwise (GFO 4, SciPy DE 4, evox 1, out of 15). Without a ground-truth classification of these hits, this does not establish specificity on dispatch; it only shows that the plateau inflated C1 on test functions.
- **Waste on dispatch (median share of repeated evaluations; waste only for deterministic, cacheable objectives):** scikit-opt DE 36%, geneticalgorithm 9.1% and DEAP 8.4%. The synthetic plateau overstated geneticalgorithm's waste, which was 41.5% there.

Post hoc sensitivity (descriptive; planned after the H52 result was known, before computing). The plan (`ITERATION52_SENSITIVITY_PLAN.md`) was committed before any of these numbers were computed; it first reproduces 19/21 from the archived cell flags.

- *Matched exposure:* each library's synthetic cells were subsampled, without replacement, to the number of completed dispatch cells (5–15). The exact distribution of the agreement count over all subsets is 17 (probability 4×10^{-5}), 18 (0.40), 19 (0.50) and 20 (0.10), with an expectation of 18.7. Only three libraries vary: Nevergrad, scikit-opt (1 positive synthetic cell of 30) and, negligibly, cma.
- *Complete libraries only* (18 with all 15 dispatch cells): agreement 17/18; positive recurrence 7/7.
- *Missing dispatch cells* (bayes_opt, bofire, Nevergrad): treating each library's missing cells as possibly positive bounds the agreement between 17 and 20 of 21.
- The same results follow from the Study 5 synthetic records, with D3x from effective hash seeds.

The minimum of both analyses equals the preregistered threshold (17), so the $\geq 80\%$ agreement survives these analyses, but without margin. Subsampling treats a library's cells as exchangeable, which they are not (functions and dimensions differ), and it does not address informative failures.

6.5 Study 5: corrected cross-process check and full traces

- **Manipulation check:** passed in all 620 valid cells; the 10 invalid cells are Nevergrad cells that fail with the same NumPy 2.5 error as in Study 1.
- **H55a supported:** D3x-diff occurred only for Nevergrad (5/20 cells) and SMAC (30/30).
- **H55b supported:** SMAC gave the identical call sequence for the same hash seed (D3x-same 0/30) and a different one for a different hash seed (30/30), with no within-process difference. The SMAC hash-seed dependence is now established within the audit, not only by the separate reproducer.
- **H55c supported:** Nevergrad also differed with the same hash seed (5/20), consistent with its clock-seeded `cma.fmin` call.
- **H55d not supported in the strict preregistered sense:** in one cell (emukit, plateau, N = 2, seed 6001) one class changed presence (C2, 0 → 1) and the C1 count changed (2 → 3). Reclassification from the traces otherwise reproduced the archived findings. The set of libraries with a finding other than C1 is unchanged: 4/9 in Study 1, 4/12 in Study 2, 8/21 in total.
- **Call sequences:** 318 of 358 Study 2 cells reproduced the archived SHA-256 bit for bit, days later and in rebuilt environments (358 = 360 cells minus 2 bofire cells that timed out in Study 2; Study 5 completed all 360). The 40 differences are explained:

- SMAC, 30 cells: the Study 2 runs had random hash seeds, and SMAC's sequence depends on the hash seed.
- emukit, 10 cells: our driver did not set `VECLIB_MAXIMUM_THREADS=1` (Apple Accelerate), which Study 2 had set. With that setting, all 10 cells matched the archive. This post hoc check shows that the number of BLAS threads alone changes the call sequence, with the same seed, versions and machine.

7 Discussion

7.1 Why this matters for comparisons

Each defect class changes a typical comparison:

- **GAdapt case.** The hidden corner evaluations most plausibly turned a validation probe into an apparent search result; the original raw data are lost, so this remains an interpretation (Section 1).
- **GFO.** Comparing GFO's `best_score` across minimizers would rank it as finding negative costs.
- **SMAC.** Seeded results cannot be reproduced by a reader who reruns the script, because Python randomizes string hashing per process.
- **scikit-opt DE and geneticalgorithm.** Spending nearly half the budget on repeated points halves the effective budget in an equal-budget comparison.
- **cma.** Over-budget evaluations make equal-budget comparisons slightly unequal.

7.2 Why cross-process checks

Within-process reruns did not detect the SMAC issue. Python's per-process hash randomization makes any iteration over a set of hashable objects whose hash depends on strings a hidden source of randomness. The audit catches this by running two fresh subprocesses with different string hashing. The launcher must not use `python -I` (or `-E`), which silently ignores `PYTHONHASHSEED`; the corrected tool uses `-s` with `PYTHONSAFEPATH=1` and records a sentinel hash per process to prove which hash seed was in effect.

8 Threats to validity

- **Sampling.** GitHub topics and star thresholds are incomplete, so we added an external source. Eligibility excluded several libraries for installation or packaging reasons, on Python 3.14 with NumPy 2.x. A strict smoke-test rule excluded gaft because it divides by zero on a constant cost function.
- **Sample selection.** The Study 1 sample was chosen by us. Only the Study 2 sample follows an external rule. The two studies also differ in budget (1000 against 200).
- **What counts as a problem.** C3 in DEAP and evotorch reflects missing bounds handling, which the libraries leave to the user or do not support. PyGAD's post-return evaluations are documented. These count under the preregistered definition. Excluding C1 and the two design cases of C3 leaves 6/21 libraries (11–52%). The strictest measure, behaviour that contradicts the documentation, gives 4/21.
- **Test functions versus real costs.** Test functions both over- and under-state problems: they made C1 look non-specific and overstated waste, and they missed the SciPy polishing behaviour. Study 4 covers one infrastructure problem class (AC dispatch) and is no guarantee for others.
- **Configuration.** We used one prespecified configuration per library through the documented interfaces and small budgets (200 or 1000). Other usage may behave differently.

- **Applicability.** R1–R3 do not apply to libraries without a reported result. C4 applies only where a maximum is documented.
- **Post hoc analyses.** The C1 sensitivity analysis, the transfer sensitivity analysis (Section 6.4) and the causal diagnoses were done after the runs and are labelled as such. The preregistered outcomes are reported unchanged.
- **D3x launcher.** In Studies 2–4 the intended hash seeds were not applied (`python -I`, Section 3). Study 5 repeated D3x with effective, recorded hash seeds for Studies 1–2 and confirmed both findings; the dispatch study (Study 4) was not repeated.
- **Numerical environment.** The number of BLAS threads changed emukit's call sequence (Section 6.5). Reproducibility checks must fix thread counts as well as seeds.
- **Archived evidence.** Studies 1–4 archived per-run counts, flags and hashes, not full call traces, so classes cannot be recomputed independently from the archive. Study 5 re-runs the test-function cells of Studies 1–2 with compressed call traces and returned results, and a script reclassifies them and compares the flags with the archived ones. It does not cover the dispatch study (Study 4), whose agreement, waste and SciPy example remain supported by the archived flags and by the disclosure reproducer (`iteration52_disclosure/`).
- **Contract violation versus documentation gap.** We separate behaviour that contradicts the documentation (4/21) from undocumented behaviour (e.g. SciPy polishing) and from documented design choices (DEAP and evotorch bounds).
- **Novelty.** All individual Study 1 findings were already known. The contribution of Study 1 is automated detection and prevalence.

9 Recommendations for practitioners

Before comparing optimizers on an expensive cost function:

1. Wrap the cost function in a recorder.
2. Check that the reported solution was evaluated, and that its reported cost matches the recorded one.
3. Count evaluations against the budget, including calls before and after the search.
4. Rerun with the same seed in two fresh processes, with fixed hash seeds (not with `python -I`) and fixed numbers of BLAS threads.
5. Measure the share of repeated evaluations.
6. Repeat the audit with your own cost function, not only with test functions.

The tool `research/integrity_audit_tool.py` performs these steps for any optimizer through a small adapter. It returns a non-zero exit code on a problem, for use in CI.

10 Data and code availability

All material is in https://github.com/owlCoder/gadapt, directory `research/`; the revision used for this manuscript is identified by its full commit hash in the submission; the aggregate outputs of Studies 1–4 record the SHA-256 of their driver code and the commit of their protocol. An immutable archive (Zenodo DOI) of that revision will be created on acceptance. Development branches: Nevergrad, GFO and SciPy (nightly) were run; for SMAC and scikit-opt the relevant source was inspected and found unchanged.

- protocols: `ITERATION49_PROTOCOL.md` to `ITERATION52_PROTOCOL.md`, and `ITERATION55_PROTOCOL.md` (Study 5, with deviations O1–O3); post hoc transfer sensitivity plan `ITERATION52_SENSITIVITY_PLAN.md`;
- eligibility and documentation quotes: `ITERATION50_ELIGIBILITY.md`;

- code: `iteration49_integrity_audit.py`, `iteration50_integrity_audit.py`, `iteration51_d3x.py`, `iteration52_dispatch_integrity.py`, `iteration52_sensitivity.py`, `iteration55_d3x_traces.py` and `iteration55_recompute.py`;
- raw outputs, with `pip freeze` per library: `results/iteration49-52/`; Study 5 with full call traces: `results/iteration55/` and the post hoc BLAS-thread check `results/iteration55_posthoc/`;
- disclosure drafts and patches: `iteration49_nevergrad_1572/`, `iteration50_disclosure/`, `iteration52_disclosure/`;
- the practitioner tool and its tests.

A Exact predicates

Notation: a run records calls k = 1..n as (x_k, f_k, phase_k), phase ∈ {search, post}; search calls are those made before the optimizer's main call returned (adapter-specific accessors called afterwards produce post calls). The box is [lo, hi]. The result is (x*, f*, e*), any of which may be missing.

- C1: x_k = lo or x_k = hi (bitwise, all coordinates). Count of such calls.
- C2: the bytes of x_k equal those of some x_j, j < k. Count; waste share = C2 / n.
- C3: some coordinate x_k,i < lo_i or > hi_i. Count.
- C4: only if the library documents a maximum M: max(0, n − M).
- C5: number of post calls.
- R1–R3 are not applicable when the library reports no x*; they are then stored as 0 together with an explicit `r_applicable = false`, and excluded from R1–R3 counts.
- R1: no call with the same shape and $|x_k,i - x^*_i| \le$ 1e-12 for all i.
- R2: applicable if f* is present and R1 = 0: with f(x*) the value of the first matching call, $|f^* - f(x^*)| >$ 1e-9 · max(1, |f(x*)|).
- R3: applicable if f* is present and there is ≥ 1 search call: with b = min of search-phase values, $f^* - b >$ 1e-9 · max(1, |b|).
- A reported f* = NaN makes both inequalities false, so R2 and R3 are not flagged; no studied library reported NaN. No study distinguishes a missing f* from a NaN f* (Study 5 stores both as NaN).
- D2: applicable if the library reports an evaluation count e*: $e^* \neq n$ (all calls, including post).
- D3: SHA-256 over all (x_k, f_k, phase_k) differs between two runs with the same seed in one process.
- D3x: the same hash differs between two fresh processes with the same seed. Non-finite values are recorded as returned; a run that raises is reported as an error and excluded from all classes. Validation coverage: the planted faults cover C1, C2, C4, R1, R2, D3 and D3x. C3, C5 and R3 were not covered by a planted fault and are supported only by the diagnosed examples in Section 6.2; D2 has neither a planted fault nor an observed example and is therefore unvalidated.

References

Jacob de Nobel, Furong Ye, Diederick Vermetten, Hao Wang, Carola Doerr, and Thomas Bäck. 2024. IOHexperimenter: Benchmarking Platform for Iterative Optimization Heuristics. *Evolutionary Computation* 32, 3 (2024), 205–210. doi:10.1162/evco_a_00342

Alejandra Duque-Torres, Claus Klammer, Stefan Fischer, Dietmar Pfahl, and Rudolf Ramler. 2025a. Assessing the strength of Metamorphic Testing applied to optimisation software—Experience from industry. *Information and Software Technology* 186 (Oct. 2025), 107807. doi:10.1016/j.infsof.2025.107807

Alejandra Duque-Torres, Claus Klammer, Stefan Fischer, Rudolf Ramler, and Dietmar Pfahl. 2025b. Metamorphic testing for optimisation: A case study on PID controller tuning. *Information and Software Technology* 188 (Dec. 2025), 107872. doi:10.1016/j.infsof.2025.107872

Martin Gruber, Stephan Lukasczyk, Florian Krois, and Gordon Fraser. 2021. An Empirical Study of Flaky Tests in Python. In *2021 14th IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 148–158. doi:10.1109/icst49551.2021.00026

Alex Gyori, Ben Lambeth, August Shi, Owolabi Legunsen, and Darko Marinov. 2016. NonDex: a tool for detecting and debugging wrong assumptions on Java API specifications. In *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE' 16)*. ACM, 993–997. doi:10.1145/2950290.2983932

Nikolaus Hansen, Anne Auger, Raymond Ros, Olaf Mersmann, Tea Tušar, and Dimo Brockhoff. 2021. COCO: a platform for comparing continuous optimizers in a black-box setting. *Optimization Methods and Software* 36, 1 (Aug. 2021), 114–144. doi:10.1080/10556788.2020.1808977

Nikolaus Hansen, Raymond Ros, Nikolas Mauny, Marc Schoenauer, and Anne Auger. 2011. Impacts of invariance in search: When CMA-ES and PSO face ill-conditioned and non-separable problems. *Applied Soft Computing* 11, 8 (Dec. 2011), 5755–5769. doi:10.1016/j.asoc.2011.03.001

Jakub Kudela. 2022. A critical problem in benchmarking and analysis of evolutionary computation methods. *Nature Machine Intelligence* 4, 12 (Dec. 2022), 1238–1245. doi:10.1038/s42256-022-00579-0

Miha Ravber, Marko Šmid, Matej Moravec, Marjan Mernik, and Matej Črepinšek. 2025. Performance Comparison of Single-Objective Evolutionary Algorithms Implemented in Different Frameworks. *Informatica* (Jan. 2025), 677–712. doi:10.15388/25-infor603

Jeremy Rounds and Upulee Kanewala. 2018. Systematic Testing of Genetic Algorithms: A Metamorphic Testing based Approach. arXiv:1808.01033 Preprint.

Diederick Vermetten, Bas van Stein, Fabio Caraffini, Leandro L. Minku, and Anna V. Kononova. 2022. BIAS: A Toolbox for Benchmarking Structural Bias in the Continuous Domain. *IEEE Transactions on Evolutionary Computation* 26, 6 (Dec. 2022), 1380–1393. doi:10.1109/tevc.2022.3189848